

Sgt

替罪羊树

引入

替罪羊树 是一种依靠重构操作维持平衡的重量平衡树。替罪羊树会在插入、删除操作后，检测树是否发生失衡；如果失衡，将有针对性地进行重构以恢复平衡。

一般地，替罪羊树不支持区间操作，且无法完全持久化；但它具有实现简单、常数较小的优点。

基本结构和操作

替罪羊树的核心操作是重构、插入和删除操作。

节点信息

替罪羊树需要存储以下信息，用于树的自平衡操作：

- 树的结构信息：
 - `id`：已使用节点数目；
 - `rt`：根节点；
 - `lc[x]`, `rc[x]`：左、右子节点；
 - `tot[x]`：以 `x` 为根的子树大小（每个节点计数为 [1](#)）；
 - `tot_active`：整个树中未删除（即 `cnt[x] != 0`）的节点的数目。

当使用替罪羊树实现平衡树时，还需要存储如下信息：

- 平衡树的节点信息：
 - `val[x]`：节点存储的值；
 - `cnt[x]`：节点存储的值的计数（可能为 `0`）；
 - `sz[x]`：以 `x` 为根的子树存储的值的计数。

为了维护节点信息，可以实现 `push_up` 操作：

▼ 参考实现

```
1 // Update node info from its children.
2 void push_up(int x) {
3     tot[x] = 1 + tot[lc[x]] + tot[rc[x]];
4     sz[x] = cnt[x] + sz[lc[x]] + sz[rc[x]];
5 }
```

应注意 `tot[x]` 和 `sz[x]` 的更新方式的不同。

重构操作

当树发生失衡时，需要对某个子树进行重构，使之尽可能平衡。重构分为两步：

- 对要重构的子树做中序遍历，将所有未删除节点存到序列中；
- 二分建树，即取中点为根，左右两侧递归地建子树，并更新节点信息。

参考实现如下：

▼ 参考实现

```
1 // Tree -> Array.
2 void flatten(int x) {
3     if (!x) return;
4     flatten(lc[x]);
5     if (cnt[x]) tmp[n_tmp++] = x;
6     flatten(rc[x]);
7 }
8
9 // Array -> Tree.
10 int build(int ll, int rr) {
11     if (ll > rr) return 0;
12     int mm = (ll + rr) / 2;
13     int x = tmp[mm];
14     lc[x] = build(ll, mm - 1);
15     rc[x] = build(mm + 1, rr);
16     push_up(x);
17     return x;
18 }
19
20 // Rebuild a subtree.
21 void rebuild(int& x) {
22     n_tmp = 0;
23     flatten(x);
24     x = build(0, n_tmp - 1);
25 }
```

建树时注意维护节点信息，包括叶子节点的信息。

单次重构的复杂度是 $O(n)$ 的，因此如果每次插入、删除时都进行重构，复杂度将难以接受。替罪羊树的核心思想就在于对重构时机的选择，进而实现了 $O(\log n)$ 的均摊复杂度。

插入操作

插入操作时，可能会引起树的失衡。为了判断树的失衡，需要引入参数 bf ，通常的选择在 $[-1, 1]$ 之间。

如果新插入的节点的深度超过了 $2 \log n$ ，其中 n 为更新后的树的大小，就需要在回溯时寻找失衡发生的节点并进行重构。此时，需要根据如下条件判断以 x 为根的子树失衡：

其中， l 和 r 分别为 x 的左、右子节点， sz_x 为以 x 为根的子树大小。

插入操作的具体步骤如下：

- 首先利用二分查找树的性质向下找到插入值的位置，下探时记录深度；
- 如果已经有节点，直接修改节点信息，否则新建节点；
- 如果新建节点过深，就需要自下而上回溯到根，更新节点信息，并记录第一个（或任意一个）子树失衡的节点；
- 如果存在失衡节点，重构失衡节点的子树。

参考实现如下：

▼ 参考实现

```

1 // Insert v into subtree of x.
2 bool insert(int& x, int v, int dep) {
3     bool check = false;
4     if (!x) {
5         x = ++id;
6         val[x] = v;
7         check = dep > std::log(tot[rt] + 1) / std::log(1 / ALPHA);
8     }
9     if (val[x] == v) {
10         if (!cnt[x]) ++tot_active;
11         ++cnt[x];
12     } else if (v < val[x]) {
13         check = insert(lc[x], v, dep + 1);
14     } else {
15         check = insert(rc[x], v, dep + 1);
16     }
17     push_up(x);
18     if (check && (tot[lc[x]] > ALPHA * tot[x] || tot[rc[x]] > ALPHA * tot[x])) {
19         rebuild(x);
20         return false;
21     }
22     return check;
23 }
24
25 // Insert v into the tree.
26 void insert(int v) { insert(rt, v, 0); }

```

注意，单次插入至多引起一次重构。如果没有新增节点或是新增节点并没有过深，又或是本次回溯过程中已经执行过重构，就不需要继续判断失衡了。多余的重构可能会导致效率损失²。回溯过程中的第一个失衡节点，就是所谓的「替罪羊」。

删除操作

删除操作的处理则非常简单。替罪羊树的删除策略是「懒删除」，即节点为空时，不移除节点，而是留待后续处理。

当然，如果树中空节点过多，树的访问效率会大大下降。因此，替罪羊树维护两个计数，整个树中未删除节点的数目和整个树实际使用的节点数目。对于选定的阈值³，当前者与后者的比值下降到 以下时，就对整个树做一次重构，重构时删除所有空节点。

▼ 参考实现

```

1 // Remove v from subtree of x.
2 bool remove(int x, int v) {
3     if (!x) return false;
4     bool succ = true;
5     if (v < val[x]) {
6         succ = remove(lc[x], v);
7     } else if (v > val[x]) {
8         succ = remove(rc[x], v);
9     } else if (cnt[x]) {
10        --cnt[x];
11        if (!cnt[x]) --tot_active;
12    } else {
13        succ = false;
14    }
15    push_up(x);
16    return succ;
17 }
18
19 // Remove v from the tree.
20 bool remove(int v) {
21     bool succ = remove(rt, v);
22     if (!tot_active) {
23         rt = 0;
24     } else if (succ && tot_active < tot[rt] * ALPHA) {
25         rebuild(rt);
26     }
27     return succ;
28 }

```

时间复杂度

大小为 n 的替罪羊树的访问节点的时间复杂度为单次 $O(\log n)$ 的， n 次插入和删除的均摊时间复杂度也是单次 $O(\log n)$ 的。

本节对替罪羊树的时间复杂度仅做简要论证，详细证明请参考原论文。

► 替罪羊树的时间复杂度的论证

平衡树操作

本节介绍用替罪羊树维护可重集的方法。

除上节介绍的操作外，其余操作均为平衡树的常见操作。但是，因为替罪羊树中可能存在空节点，这些操作也需要相应调整。

查询排名

利用二分查找树的性质向下查找节点位置，过程中记录路径左侧存储的值的数目即可。

▼ 参考实现

```

1 // Find the rank of v, i.e., #{val < v} + 1.
2 int find_rank(int v) {
3     int res = 0;
4     int x = rt;
5     while (x && val[x] != v) {
6         if (v < val[x]) {
7             x = lc[x];
8         } else {
9             res += sz[lc[x]] + cnt[x];
10            x = rc[x];
11        }
12    }
13    return res + sz[lc[x]] + 1;
14 }

```

根据排名查询值

利用节点记录的子树存储值的数目信息向下查找即可。注意可能存在计数为零的节点。

▼ 参考实现

```

1 // Find the k-th smallest element.
2 int find_kth(int k) {
3     if (k <= 0 || sz[rt] < k) return -1;
4     int x = rt;
5     for (;;) {
6         if (k <= sz[lc[x]]) {
7             x = lc[x];
8         } else if (k <= sz[lc[x]] + cnt[x]) {
9             return val[x];
10        } else {
11            k -= sz[lc[x]] + cnt[x];
12            x = rc[x];
13        }
14    }
15    return -1;
16 }

```

查询前驱、后继

以上两种功能结合即可。

▼ 参考实现

```

1 // Find predecessor.
2 int find_prev(int x) { return find_kth(find_rank(x) - 1); }
3
4 // Find successor.
5 int find_next(int x) { return find_kth(find_rank(x + 1)); }

```

如果想直接实现，应注意处理计数为零的节点。

参考实现

本节的最后，给出模板题 [普通平衡树](#) 的参考实现。

► 参考实现

参考资料

- Galperin, Igal, and Ronald L. Rivest. "Scapegoat trees." Proceedings of the fourth annual ACM-SIAM Symposium on Discrete algorithms. 1993.
 - [Scapegoat Tree - Wikipedia](#)
 - [替罪羊树 - riteme 的博客](#)
-

1. 也可以只统计未删除节点数目，此时不再需要统计 `tot_active`，而需要统计所有占用节点数目 `tot_max`，代码相应调整即可。↩
 2. 根据后文的复杂度分析可知，这些效率损失仅意味着更大的常数因子，而复杂度依然是正确的。因为判断树深可能会涉及较多的浮点数对数运算，不判断树深只判断失衡的代码在某些数据中可能更快。↩
 3. 不必与上文插入操作时选取的参数相同。尽管原论文做了这样的假定，但是选取不同的参数只会导致单次操作的复杂度中常数项的变化，整体复杂度依然是正确的。↩
 4. 按原文定义，指未删除节点的数目，故而只能保证树高不超过 $\log n$ ，这称为弱 $\log n$ -高度平衡。此处没有细究该常数项的差异。↩
 5. 有些文章会简单分析成 $O(n \log n)$ 次插入对应一次重构，故而均摊复杂度为 $O(\log n)$ 。这样的思路可以辅助理解均摊复杂度为什么正确，但并不严谨。这是因为，一次插入可能对应着多个祖先节点的重构，故而当节点 x 发生重构时，子树内未引起重构的节点数目并不显然是 $O(n)$ 的。↩
-

本页面最近更新：2025/3/20 21:13:06, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[Oxis-cn](#), [lr1d](#), [Backlight](#), [c-forrest](#), [countercurrent-time](#), [Early0v0](#), [Enter-tainer](#), [ezoixx130](#), [iamtwz](#), [ksyx](#), [NachtgeistW](#), [SukkaW](#), [Tiphereth-A](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用